Lab 5: Merge Sort vs. Quick Sort

Due: 11:59pm on April 20, 2025 (Sunday)

Worst-case running time complexity of merge sort is O(n log n) whereas that of quick sort is O(n^2). With most canonical or typical implementations, both have O(n log n) for average case and best case.

However, quick sort is typically considered to be among the fastest in-memory sorting algorithms in practice. (*Taking only comparison-based sorting algorithms into consideration.*)

That's interesting!
Let's see how merge sort and quick sort are different while trying to answer the following question.

What is the most efficient way to sort a million 32bit integers?

When you hear this kind of question, you should ask a couple of questions back.

- What does efficiency mean?
- Can I assume that the input data is totally random or is there any possibility of the input data being almost sorted already?

Given that you have only five choices for sorting (bubble sort, selection sort, insertion sort, merge sort, and quick sort), which algorithm would you choose given the different possible answers for those questions?

Moving on, let's first look at what difference the pivot value makes in Quick Sort.

Download `QuickSort.java` and `QuickSortImproved.java` files along with `Stopwatch.java` file from Canvas and run them.

What is the output of QuickSort?

What is the output of QuickSortImproved?

Now, let's comment out the for loop that sets the values of the array and uncomment the commented-out for loop in both files and run them again.

What is the output of QuickSort?

What is the output of QuickSortImproved?

Do you see any difference? What is the major difference between QuickSort and QuickSortImproved?
Now, let's see what we can do to improve merge sort. In the advanced sorting lecture for merge sort, you noticed that we created two sub arrays (one for left and one for right) and sort them recursively as follows.

```
public static int[] mergeSort(int[] unsorted) {
    if (unsorted.length <= 1) {
        return unsorted;
    }
    int[] left = new int[unsorted.length / 2];
    System.arraycopy(unsorted, 0, left, 0, left.length);
    int[] right = new int[unsorted.length - left.length];
    System.arraycopy(unsorted, left.length, right, 0, right.length);
    left = mergeSort(left);
    right = mergeSort(right);
    return merge(left, right);
}
```

This means that every time we call the method, two new sub arrays are being created. In addition, merge method creates a new array to which all the values from left and right arrays are copied over.

Question:

You have a concern about your limited memory. Can you implement merge sort in a way that it does not create sub arrays?

```
public static void mergeSort(int[] from) {
    // create a new array
    int[] to = new int[from.length];
    mergeSort(from, to, 0, from.length-1);
}
private static void mergeSort(int[] from, int[] to, int left, int right) {
    if(_____________________________) return; // base case

    // find midpoint
    int mid = left + (right - left) / 2;
    // sort left half recursively
    mergeSort(from, to, left, mid);
    // sort right half recursively
    mergeSort(from, to, mid+1, right);
    // merge them
    merge(from, to, left, mid+1, right);
}

/* instead of creating arrays over and over,
 * use only two arrays (from and to)
 * 1. leftPos: starting point of left half
 * 2. rightPos: starting point of right half
 * 3. rightBound: upper bound of right half
 */
public static void merge(int[] from, int[] to, int leftPos, int rightPos, int
rightBound) {
    // upper bound of left half
    int leftBound = _____________________________;
    // index of to array, starting point of left half
    int toIndex = _____________________________;
    // total number of items to examine
    int numOfItems = rightBound - _____________ + _____________;
```

```
    // both left half and right half have items
    while (leftPos <= _____________ && rightPos <= _____________) {
        if (from[leftPos] <= from[_____________________]) {
            to[toIndex++] = from[_____________________];
        } else {
            to[toIndex++] = from[_____________________];
        }
    }

    // Copy rest of left half
    while (leftPos <= _____________________) {
        to[toIndex++] = from[_____________________];
    }
    // Copy rest of right half
    while (rightPos <= _____________________) {
        to[toIndex++] = from[_____________________];
    }

    // Post process that needs to be done.
    // Can you see what this is supposed to do?
    for (int i = 0; i < numOfItems; i++, rightBound--) {
        from[_____________________] = to[_____________________];
    }
}
```

Download the `MergeSort.java` file and complete the code. You may use the template provided above to help write the code.

Once you are done, try to run MergeSort and QuickSort. **Make sure that you are using the original for loop in QuickSort class to compare them properly.**
Also, pay attention to the value of the SIZE variable!

1. What is the output of MergeSort?

2. What is the output of QuickSort?

3. Which one is better? Can you think why?

As you may be able to see from the code, the following ensures that the values of the input array are randomized.

```
private static final int SIZE = 100000;
private static Random rand = new Random();

public static void main(String[] args) {
    int[] array = new int[SIZE];
    for (int i = 0; i < SIZE; i++) {
        array[i] = rand.nextInt();
    }

    ……
}
```

Now, **comment the original for loop and uncomment the commented for loop in MergeSort and QuickSort!**
Once you are done, try to run them again.

1. What is the output of MergeSort?

2. What is the output of QuickSort?

After setting the SIZE value to be **1,000,000 in both files**, run them again. (If you are getting a stack overflow exception, you can use the **JVM argument, "-Xss"**, to increase the default stack size.)

1. What is the output of MergeSort?

2. What is the output of QuickSort?

Additionally, can you think of some other ways to improve the quick sort?

If so, I would encourage you to implement those ideas on your own.

Finally, let's try to answer the original question!

Which sorting algorithm (given you have a choice between merge sort and quick sort) would you choose to sort a million 32bit integers?

Submit your `MergeSort.java` file using Autolab (https://autolab.andrew.cmu.edu).
Do not zip it, and do not use package statements.